Week 22 — Extended reading

Constrained MDPs, CPO/FOCOPS/P3O, Lagrangian Safety

STUDY-RL

May 21, 2026

Paper 1 — "Benchmarking Safe Exploration in Deep RL" (PPO-Lagrangian)

Authors: Alex Ray, Joshua Achiam, Dario Amodei. **Year:** 2019. **Venue:** OpenAI tech. report. **arXiv:** 1910.01708.

Summary. Introduces *Safety Gym*, a suite of MuJoCo robotics tasks with constraint signals (e.g. entering hazards, contacting fragile objects), and benchmarks PPO-Lagrangian and CPO on them. The paper formalises the CMDP API for deep RL, gives canonical hyperparameters, and shows that simple primal-dual PPO already matches or beats CPO on most tasks when carefully tuned — a foundational empirical reference.

Why it matters for this week. It is the canonical implementation template for PPO-Lagrangian. Our `project/agent.py` follows the same architecture (separate V_r, V_c heads; combined advantage; projected multiplier update) and its decision to keep the rollout small for the dual loop comes directly from this paper.

Annotated key passage.

> "Despite the simplicity of the Lagrangian approach, getting it to work reliably in practice required several engineering decisions: separate cost critics, advantage standardisation *only* on the reward, and a learning rate for λ that is one to two orders of magnitude smaller than the policy learning rate to avoid oscillation."

Commentary. Every implementer rediscovers this trio of tricks within their first week of hacking on safe RL — the paper saves you that week. The small dual-LR is essentially a poor-person's PID damping; the same effect is later formalised by Stooke et al. as PID-Lagrangian.

Paper 2 — "Constrained Policy Optimization" (CPO)

Authors: Joshua Achiam, David Held, Aviv Tamar, Pieter Abbeel. **Year:** 2017. **Venue:** ICML. **arXiv:** 1705.10528.

Summary. The first deep-RL algorithm with a per-iteration near-feasibility guarantee. CPO extends TRPO by adding a cost-constraint to the trust-region QP, deriving a closed-form dual solution (Theorem 4.1 in the paper). It also describes a *recovery step* when the linearised constraint cannot be satisfied inside the trust region.

Why it matters. It is the theoretical reference point for all later constrained RL work; FOCOPS, P3O, and PID-Lag all benchmark against it. Even if you choose not to implement Fisher-vector products on a single GPU, you must understand CPO to read modern papers.

Annotated key passage.

> "Theorem 1 (CPO performance bound). For any policies π' and π, $J_c(\pi') \leq J_c(\pi) + \frac{1}{1-\gamma}\mathbb{E}_{s\sim d^\pi, a\sim\pi'}[A_c^\pi(s,a)] + \frac{\sqrt{2}\gamma\epsilon_c^\pi}{(1-\gamma)^2}\sqrt{\frac{1}{2}\bar{D}_{\mathrm{KL}}(\pi'\|\pi)}$."

Commentary. This inequality is the central object of CPO. The $1/(1-\gamma)^2$ scaling tells you the long-horizon penalty for trust-region slack — short-horizon tasks tolerate larger δ. The $\sqrt{\bar{D}_{\mathrm{KL}}}$ dependence is what makes the trust-region argument non-vacuous, and exactly why TRPO's KL constraint pulls double duty here.

Paper 3 — "FOCOPS"

Authors: Yiming Zhang, Quan Vuong, Keith Ross. **Year:** 2020. **Venue:** NeurIPS. **arXiv:** 2002.06506.

Summary. Solves the CPO problem in two phases: (i) compute the non-parametric optimal policy in closed form (KL-regularised reward maximisation with a cost multiplier); (ii) project onto the parametric family by KL minimisation. Replaces all Fisher–vector products with first-order updates.

Why it matters. Best-in-class fit to a single-GPU budget. Conjugate-gradient steps inside CPO can dominate the wall-clock cost on deep policies; FOCOPS removes them while retaining a CPO-style theoretical worst-case bound (in expectation, not worst-case-per-state).

Annotated key passage.

> "Lemma 1. Given fixed multipliers ν, λ, the optimal solution to the relaxed problem in non-parametric policy space is $\pi^\star(a|s) \propto \pi_k(a|s)\exp(\frac{1}{\lambda}(A_r - \nu A_c))$."

Commentary. This is the safe-RL analogue of the energy-based reweighting that appears in advantage-weighted regression (AWR) and KL-control LLM finetuning. The same trick keeps appearing because it is just the Boltzmann form of an entropy-regularised reward-max problem.

Paper 4 — "P3O" (Penalised PPO)

Authors: Linrui Zhang, Quan Vuong, Keith Ross. **Year:** 2022. **Venue:** IJCAI. **arXiv:** 2205.11814.

Summary. Replaces the saddle-point with an exact-penalty formulation: $L_r - \kappa[\hat{J}_c - d]_+$. Under mild constraint qualification, sufficiently large κ yields a feasible minimiser of the penalised problem that is also the constrained optimum.

Why it matters. Often a one-line drop-in into PPO. The cost is that κ becomes a hyperparameter to tune; the benefit is that there is no dual variable to maintain, and the loss is smoother to optimise.

Annotated key passage.

> "Proposition 4.1 (exact penalty). Let $\kappa^\star = \lambda^\star + \rho$ for any $\rho > 0$, where $\lambda^\star$ is the optimal Lagrange multiplier of the original CMDP. Then for any $\kappa \geq \kappa^\star$, every local minimiser of $L_r - \kappa[\hat{J}_c - d]_+$ is feasible."

Commentary. This is a clean application of the exact-penalty theorem of nonlinear programming. The catch: you do not know $\lambda^\star$ a priori, so κ is over-set in practice — which is fine for the penalty argument but sometimes destabilises the policy gradient.

Paper 5 — "PID-Lagrangian"

Authors: Adam Stooke, Joshua Achiam, Pieter Abbeel. **Year:** 2020. **Venue:** ICML. **arXiv:** 2007.03964.

Summary. Treats the dual variable as the output of a PID controller driven by the constraint error $\hat{J}_c - d$. The integral term removes steady-state bias; the derivative term damps oscillations. Empirically matches CPO with a much simpler implementation.

Why it matters. Best-in-class *practical* Lagrangian implementation as of 2024–2025 (see the May 2025 empirical-study paper arXiv:2510.17564, which reports PID-Lag-with-cost-normalisation as top-2 across modern Safety-Gymnasium tasks).

Annotated key passage.

> "We discovered that the Lagrangian's instability is best understood as a classical control problem. The optimisation alternates between two coupled loops (policy, multiplier) with different timescales — the same structure that motivates PI control in process engineering."

Commentary. This re-framing is genuinely useful: once you start seeing Lagrangian RL as a control problem, the bag of damping tricks (moving averages, leaky integrators, anti-windup) becomes available off-the-shelf. The exercise on PID-Lagrangian (workbook Ex. 8) sets up the mechanics.

Sourced from. `research/safety_rl_sota.md`, sections 1 (Constrained MDPs and Lagrangian methods) and 3 (risk-sensitive RL).